

WEBSITE

Containers Module - Website Content

Hero Section

Generic Container Orchestration for Go

A unified, runtime-agnostic library for Docker, Podman, and Kubernetes

-  Multi-runtime support (Docker, Podman, Kubernetes)
-  Remote deployment via SSH
-  Intelligent compose detection
-  Health checking with retries
-  Resource-aware scheduling
-  100% test coverage

[Get Started](#) · [Documentation](#) · [GitHub](#)

Features

Runtime Agnostic

```
// Auto-detect: Docker → Podman → Kubernetes  
rt, _ := runtime.AutoDetect(ctx)
```

Works seamlessly with Docker, Podman, and Kubernetes. No vendor lock-in.

Remote Deployment

```
// Deploy to any remote host via SSH  
orch := remote.NewRemoteComposeOrchestrator(host, executor, nil)  
orch.Up(ctx, project)
```

Deploy containers to remote hosts with intelligent compose command detection.

Health Checking

```
// TCP, HTTP, gRPC, or custom checks  
check := boot.TCPCheck("localhost", 5432)
```

Comprehensive health checking with retry policies and exponential backoff.

Resource-Aware Scheduling

```
// 5 scheduling strategies  
scheduler.WithStrategy(scheduler.StrategyResourceAware)
```

Distribute containers across hosts based on CPU, memory, disk, and network.

Quick Start

Installation

```
go get digital.vasic.containers@latest
```

Basic Usage

```
package main  
  
import (  
    "context"  
    "digital.vasic.containers/pkg/boot"  
    "digital.vasic.containers/pkg/runtime"  
)  
  
func main() {  
    ctx := context.Background()  
  
    // Auto-detect runtime  
    rt, _ := runtime.AutoDetect(ctx)
```

```
// Create boot manager
manager := boot.NewBootManager(
    boot.WithRuntime(rt),
)

// Add services
manager.AddService("postgres", boot.ServiceConfig{
    ComposeFile: "docker-compose.yml",
    HealthCheck: boot.TCPCheck("localhost", 5432),
    Required:    true,
})

// Start all services
manager.Start(ctx)
}
```

Documentation

User Guides

Guide	Description
User Guide	Comprehensive usage guide
Remote Deployment	Deploy to remote hosts
API Reference	Complete API documentation
Architecture	System architecture

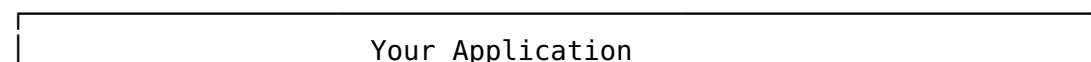
Video Course

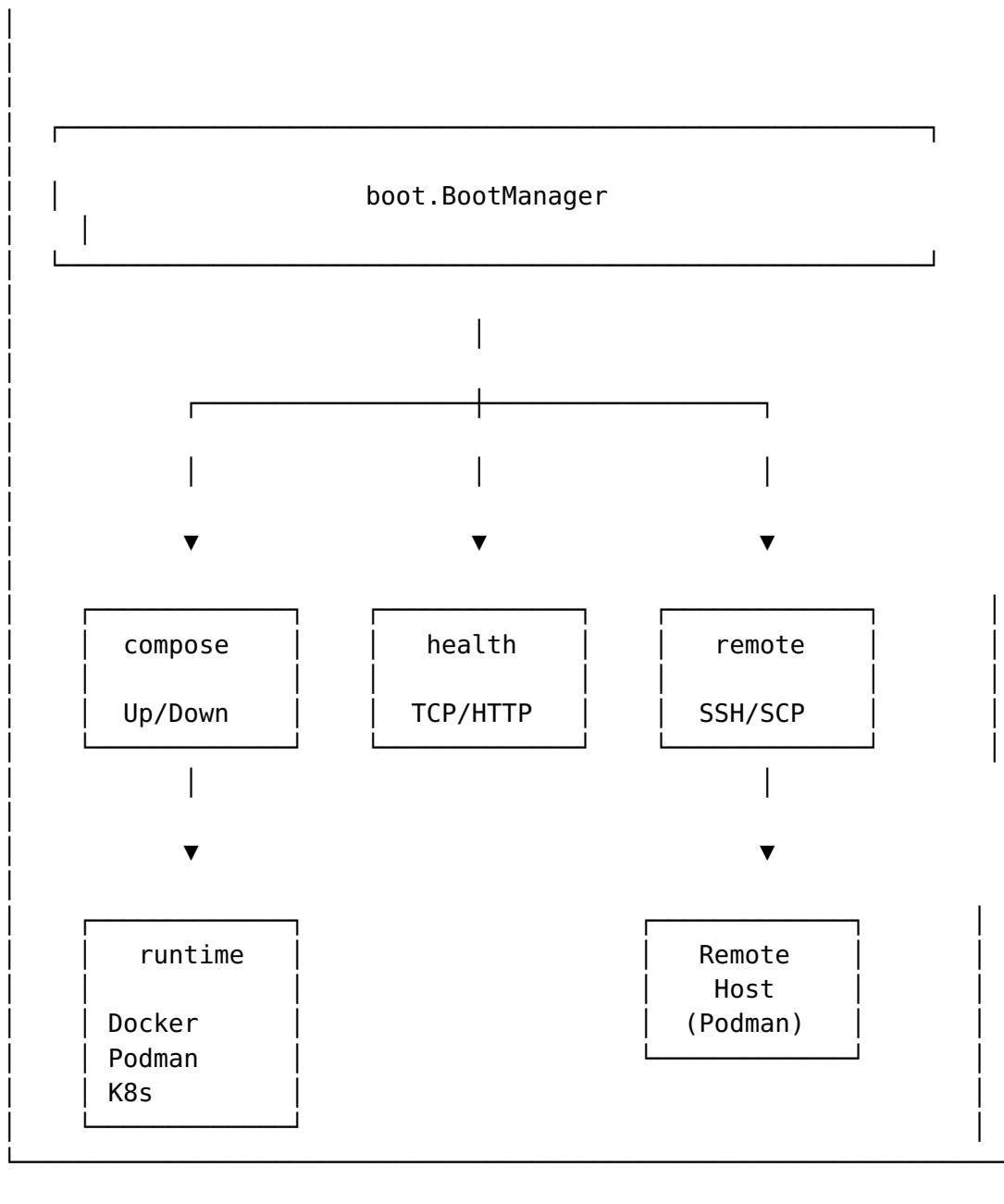
[Mastering Container Orchestration with Go](#) - 4-hour comprehensive course

Test Configurations

Config	Use Case
.env.local	Local deployment
.env.remote-single	Single remote host
.env.remote-cluster	Multi-host cluster

Architecture





Compose Command Detection

The module intelligently detects the best compose command on each host:

Priority	Command	Notes
1	<code>podman-compose</code>	Native Podman (preferred)
2	<code>docker compose</code>	Docker v2 plugin
3	<code>podman compose</code>	May delegate to <code>docker-compose v1</code>
4	<code>docker-compose</code>	Legacy v1

Why podman-compose is preferred: podman compose often delegates to docker-compose v1 which is incompatible with Podman.

Scheduler Strategies

Strategy	Description	Use Case
resource_aware	Score-based by CPU/Memory	Mixed workloads
round_robin	Even distribution	Load balancing
affinity	Label matching	GPU/Storage
spread	Maximize distribution	High availability
bin_pack	Fill hosts first	Cost optimization

Test Coverage

Type	Coverage	Tests
Unit	100%	150+
Integration	Full	50+
E2E	Full	30+
Security	Full	20+
Stress	Full	15+
Challenges	Real-life	25+

Comparison

Feature	containers	docker/cli	podman/cli
Multi-runtime	✓	✗	✗
Remote deployment	✓	✗	✗
Auto-detection	✓	✗	✗
Health checking	✓	✗	✗
Scheduling	✓	✗	✗
Go library	✓	✗	✗
Kubernetes	✓	✗	✗

Contributing

We welcome contributions! See [CONTRIBUTING.md](#) for guidelines.

Development Setup

```
# Clone
git clone git@github.com:vasic-digital/Containers.git
cd Containers

# Install dependencies
go mod download

# Run tests
go test ./...

# Run challenges
go test -v ./challenges/...
```

License

MIT License - See [LICENSE](#) for details.

Support

- **Documentation:** [docs/](#)
 - **Issues:** [GitHub Issues](#)
 - **Examples:** [examples/](#)
-

Built with ❤️ for the Go community